B.TECH CSE | YEAR III, SEMESTER V | BCS501

Formal Language and Automata Theory

Detailed syllabus-mapped study notes

Three modules | Definitions | Worked constructions | Exam and viva review

Prepared from the uploaded BCS501 syllabus. Topic order and terminology are retained.

Module map and study route

Finite Automata

- Finite State Systems.
- Basic definitions.
- Deterministic Finite Automata (DFA).
- Non-Deterministic Finite Automata (NDFA).
- Equivalence of DFA and NDFA.
- Finite Automata with ε-moves.

Regular Expressions

- Regular expressions.
- Equivalence of Finite Automata and Regular Expressions.
- Conversion of Regular Expressions to Finite Automata and vice versa.
- Arden’s Theorem.

Introduction to Machines

- Concept of basic machine.
- Properties and limitations of Finite State Machines (FSM).
- Moore Machines.
- Mealy Machines.
- Equivalence of Moore and Mealy Machines.

Module map and study route

Properties of Regular Sets

- Pumping Lemma for Regular Sets.
- Applications of Pumping Lemma.
- Closure properties of Regular Sets.
- Myhill-Nerode Theorem.
- Minimization of Finite Automata.
- Minimization Algorithms.

Grammars

- Definition of Grammar.
- Regular Grammar.
- Context Free Grammar (CFG).
- Context Sensitive Grammar (CSG).
- Ambiguity in Grammars.
- Reduced forms.
- Removal of useless symbols and unit productions.
- Chomsky Normal Form (CNF).
- Greibach Normal Form (GNF).

Module map and study route

Pushdown Automata

- Introduction to Pushdown Machines.
- Acceptance of PDA.
- Conversion of PDA to CFG and CFG to PDA.
- Applications of Pushdown Machines.

Turing Machines

- Deterministic Turing Machines.
- Non-Deterministic Turing Machines.
- Design of Turing Machines.
- Halting Problem.
- Post Correspondence Problem (PCP).

Chomsky Hierarchy and Computability

- Chomsky Hierarchy of Grammars.
- Unrestricted Grammars.
- Context Sensitive Languages.
- Relationship among language classes.
- Computability concepts.
- Primitive Recursive Functions.

Module I: Finite Automata and Regular Expressions

15 syllabus topics | definitions, constructions, proofs and review



Finite State Systems.

Simple explanation

A finite-state system has a limited set of modes. It reads one input item at a time, changes mode, and can report whether the input satisfies a rule. A turnstile, protocol controller, or text-pattern recognizer can be described this way.

Formal / technical definition

A deterministic finite-state system is a transition system (Q, Sigma, delta, q0) with finite state set Q, input alphabet Sigma, initial state q0, and transition function delta: Q x Sigma -> Q. An acceptor adds a set F of accepting states.

Mental model

A finite-state system compresses the relevant past into one of finitely many states.

Notation to remember

State = all history that can affect future decisions. The machine does not retain the full input; it retains only its current state.

Finite State Systems.

How to work with this idea

- Choose the finite set of situations that matter to the task.
- Name each situation as a state and mark the start state.
- For each state and input symbol, define the next state.
- For recognition, mark the states that mean the rule has been satisfied.

```
E --1--> O; O --1--> E; both states loop on 0. Accept E for an even number of 1s.
```

Solved example / construction

Parity of the number of 1s needs only two states: E (even so far) and O (odd so far). Reading 1 toggles E <-> O; reading 0 leaves the state unchanged. The final state tells parity without storing the count.

Finite State Systems.

Common mistakes

- A state is not necessarily a position in the input.
- The input is processed left to right; transitions do not skip symbols unless an epsilon move is explicitly allowed.
- A finite-state machine has finitely many states even if its input strings can be arbitrarily long.

Exam / viva prompts

- Why is parity a finite-state property? Because two states summarize every prefix and have identical future behavior within each parity class.
- What does the state summarize? Only the information needed to decide all possible continuations.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

A finite-state system compresses the relevant past into one of finitely many states.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Basic definitions.

Simple explanation

An alphabet is a set of symbols. A string is a finite sequence of them. A language is any set of strings over the alphabet. An automaton is a precise device for deciding which strings belong to a language.

Mental model

Keep the machine tuple, transition function, and language accepted by the machine distinct.

Formal / technical definition

Sigma is a finite alphabet; Sigma* is the set of all finite strings including epsilon; Sigma+ = Sigma* - {epsilon}. A DFA is (Q, Sigma, delta, q0, F), where delta: Q x Sigma -> Q. It accepts w when delta*(q0,w) is in F.

Notation to remember

delta* extends one-symbol transitions: delta*(q,epsilon)=q; delta*(q,xa)=delta(delta*(q,x),a). L(M)={w in Sigma*: delta*(q0,w) in F}.

Basic definitions.

How to work with this idea

- List the alphabet and the empty string separately: epsilon has length 0.
- Define states, start state, and accepting states.
- Trace a string symbol by symbol, applying delta each time.
- Accept exactly when the state after the final symbol is in F.

```
State | a | b
->q0 | q1 | q0
*q1 | q1 | q0
```

Solved example / construction

Let Sigma={a,b}. For L={w: w ends in a}, use q0='not currently ending in a' and q1='ends in a'. Start q0, F={q1}; delta(q0,a)=q1, delta(q0,b)=q0, delta(q1,a)=q1, delta(q1,b)=q0. Thus aba is accepted; abb is rejected.

Basic definitions.

Common mistakes

- epsilon is a string; the empty set is a language, not a string.
- A transition function for a DFA must specify one next state for every state-symbol pair.
- Acceptance is checked after all input is consumed, not whenever an accepting state is visited.

Exam / viva prompts

- Distinguish Sigma*, Sigma+, and epsilon.
- What is delta*? The extended transition function that consumes a whole string.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Keep the machine tuple, transition function, and language accepted by the machine distinct.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Deterministic Finite Automata (DFA).

Simple explanation

A DFA never has to guess. At every step, the current state and next symbol determine exactly one next state. Every input therefore has one unique path.

Formal / technical definition

A DFA is M=(Q,Sigma,delta,q0,F) with total function delta:Q x Sigma -> Q. The extended function delta* recursively processes words; w is accepted iff delta*(q0,w) is in F.

Mental model

A DFA has one computation path and a complete transition function.

Notation to remember

Total means every cell of the state-by-symbol transition table has a value. A missing transition can be represented by a rejecting trap state.

Deterministic Finite Automata (DFA).

How to work with this idea

- Translate the condition into a small memory summary, such as last symbol or parity.
- Create a state for each distinguishable summary.
- Fill every transition-table cell.
- Mark start and accepting states, then test boundary strings such as epsilon and one-symbol inputs.

State	a	b
->q0	q1	q0
*q1	q1	q0

Solved example / construction

For strings over {a,b} ending in a, use q0 and q1 as in the table on this slide. Trace aba: q0 -a-> q1 -b-> q0 -a-> q1, so accept. Trace epsilon: remain at q0, so reject.

Deterministic Finite Automata (DFA).

Common mistakes

- Do not leave transitions undefined in a DFA.
- Do not mark an intermediate state as acceptance unless the input ends there.
- Check epsilon explicitly: it is accepted exactly when q0 is accepting.

Exam / viva prompts

- Why is a DFA deterministic? delta returns one state, not a set of alternatives.
- How do you totalize an incomplete DFA? Add a non-accepting sink and route missing transitions to it.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

A DFA has one computation path and a complete transition function.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Non-Deterministic Finite Automata (NDFA).

Simple explanation

An NDFA may have several possible next states for a state-symbol pair, or none. It accepts if at least one possible path ends in an accepting state after consuming the whole input.

Mental model

NDFA acceptance is existential: some complete path must accept.

Formal / technical definition

An NDFA is (Q,Sigma,delta,q0,F), where delta:Q x Sigma -> 2^Q. Extended transitions propagate a set of current states; acceptance occurs when that set intersects F.

Notation to remember

A path is a sequence of transitions labeled by input symbols. Nondeterminism is an acceptance definition, not random behavior or extra physical power for regular languages.

Non-Deterministic Finite Automata (NDFA).

How to work with this idea

- For a pattern with several possible matching positions, let branches represent alternative guesses.
- At each symbol, update the active state set to the union of all destinations.
- After the last symbol, accept if any active state belongs to F.
- If no active states remain, that branch contributes no acceptance.

```
State-set trace for 'aab': {q0}->{q0,q1}->{q0,q1}->{q0,q2}; q2 in the set, so accept.
```

Solved example / construction

Recognize strings over {a,b} containing substring ab: q0 can stay in q0 while reading any prefix, and may branch on an a to q1; q1 on b reaches accepting q2. The input ab has a successful path q0 -a-> q1 -b-> q2. A string without ab has none.

Non-Deterministic Finite Automata (NDFA).

Common mistakes

- All branches must consume the input; reaching F early alone is insufficient.
- The state-set simulation uses union, not intersection.
- One accepting path is enough; one rejecting path does not cancel it.

Exam / viva prompts

- Can an NDFA have zero transitions on a symbol? Yes; that branch dies.
- How many accepting paths are required? At least one.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

NDFA acceptance is existential: some complete path must accept.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Equivalence of DFA and NDFA.

Simple explanation

A DFA and an NDFA can look different internally yet accept exactly the same strings. Every NDFA can be simulated by a DFA whose state records the set of NDFA states that could currently be active.

Mental model

DFA and NDFA recognize the same class: regular languages.

Formal / technical definition

For N=(Q,Sigma,delta,q0,F), the subset construction builds D=(2^Q,Sigma,Delta,{q0},F_D), with Delta(S,a)=union over q in S of delta(q,a), and F_D={S:S intersects F}. Remove unreachable subsets afterward.

Notation to remember

There can be up to 2^|Q| subset-states, although often far fewer are reachable. The empty subset is a valid dead state.

Equivalence of DFA and NDFA.

How to work with this idea

- Start with the subset {q0}.
- For each discovered subset S and symbol a, compute the union of destinations.
- Add any new subset to the worklist and repeat until no new subsets appear.
- Mark subsets containing at least one original final state.

```
NDFA active set {q0,q1} on b -> delta(q0,b) union delta(q1,b) = {q2}.
```

Solved example / construction

If q0 on a can reach {p,r}, then the DFA transition from {q0} on a is {p,r}. If p on b reaches f and r has no b-move, the next subset is {f}; it is accepting because it intersects F.

Equivalence of DFA and NDFA.

Common mistakes

- Do not create only singleton subsets; combinations are essential.
- Do not mark a subset final only when all members are final. The test is nonempty intersection.
- Account for the empty subset when transitions disappear.

Exam / viva prompts

- Why does subset construction work? It maintains exactly the set of NDFA states reachable after each prefix.
- What is the worst-case DFA state count? 2^n for an n-state NDFA.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

DFA and NDFA recognize the same class: regular languages.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Finite Automata with ε-moves.

Simple explanation

An epsilon move changes state without consuming an input symbol. It is useful for joining small automata into a larger pattern machine.

Formal / technical definition

An epsilon-NFA has delta:Q x (Sigma union {epsilon}) -> 2^Q. The epsilon-closure E(S) contains every state reachable from S using zero or more epsilon transitions. On symbol a, update to E(move(E(S),a)).

Mental model

Epsilon transitions alter control state for free; closure accounts for every free move.

Notation to remember

Zero epsilon moves are allowed, so S is a subset of E(S). Epsilon is not an input symbol and does not increase the input length.

Finite Automata with ε-moves.

How to work with this idea

- Compute epsilon-closure of the start state; use it as the initial active set.
- For each input symbol, take all matching symbol-moves from the active closure.
- Take epsilon-closure of the destinations.
- Accept if the final active closure intersects the final-state set.

```
s --epsilon--> p --a--> f*. Initial closure {s,p}; after a: {f}.
```

Solved example / construction

Suppose s --epsilon--> p, p --a--> f, and f is final. The closure of {s} includes p; reading a reaches f, so the one-letter input a is accepted. No epsilon was consumed from the input.

Finite Automata with ε-moves.

Common mistakes

- Do not place epsilon in the input alphabet when it denotes a no-input transition.
- Do not forget closure again after consuming a symbol.
- Closure is transitive: follow chains, not just one epsilon edge.

Exam / viva prompts

- What is epsilon-closure? All states reachable by zero or more epsilon moves.
- Can epsilon-NFA recognize more than DFA? No; it still recognizes regular languages.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Epsilon transitions alter control state for free; closure accounts for every free move.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Regular expressions.

Simple explanation

A regular expression is a compact pattern description. It combines simple symbols with choice, concatenation, and repetition to denote a language.

Formal / technical definition

Over Sigma, epsilon denotes {epsilon}, empty-set denotes {}, a in Sigma denotes {a}; if r,s are expressions, r|s denotes union, rs concatenation, and r* zero-or-more repetition. Parentheses group operations.

Mental model

A regular expression denotes a set of strings; it is not a machine execution trace.

Notation to remember

Precedence convention: star first, concatenation next, union last. r+ often means one-or-more and is shorthand r r*; verify the syllabus convention if a symbol is used.

Regular expressions.

How to work with this idea

- Identify the required strings and their boundaries.
- Use union for alternatives and concatenation for ordered parts.
- Use star only for a portion that repeats zero or more times.
- Test the expression against epsilon, shortest accepted strings, and near-miss strings.

```
(a|b)*a = all strings over {a,b} whose final symbol is a.
```

Solved example / construction

Over {0,1}, (0|1)*01 denotes all binary strings ending in 01. The prefix may be any binary string; the final two symbols are fixed. It accepts 01, 101, and 0001, but not 10.

Regular expressions.

Common mistakes

- r* includes epsilon even when r cannot produce epsilon.
- A dot or space may be notation for concatenation, not a literal character.
- Use parentheses to make scope explicit, such as (a|b)c.

Exam / viva prompts

- What does r|s mean? Union of the two languages.
- Why does (a|b)* include epsilon? Zero repetitions are allowed.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

A regular expression denotes a set of strings; it is not a machine execution trace.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Equivalence of Finite Automata and Regular Expressions.

Simple explanation

Finite automata and regular expressions are two descriptions of the same regular patterns: one is operational, the other algebraic.

Formal / technical definition

For every regular expression r, an equivalent finite automaton A exists with L(A)=L(r); for every finite automaton A, a regular expression r exists with L(r)=L(A). This characterizes regular languages.

Mental model

Every finite automaton and every regular expression has a counterpart with the same language.

Notation to remember

Equivalence means equality of languages, not identical syntax, number of states, or transition structure.

Equivalence of Finite Automata and Regular Expressions.

How to work with this idea

- To compare two descriptions, determine whether they accept exactly the same strings.
- For RE-to-automaton, build fragments for symbols and combine using union, concatenation, and star.
- For automaton-to-RE, eliminate intermediate states while preserving path labels.
- Use short-string tests as checks, not as a proof of equivalence.

Solved example / construction

The DFA for strings ending in 01 and the RE (0|1)*01 are equivalent because both accept precisely those binary strings whose last two symbols are 0 then 1.

Equivalence of Finite Automata and Regular Expressions.

Common mistakes

- Matching several examples does not prove equivalence.
- Equivalent descriptions may have very different sizes.
- Do not confuse equivalence of machines with equality of their state sets.

Exam / viva prompts

- State the theorem in both directions.
- What class is recognized/denoted? Exactly the regular languages.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Every finite automaton and every regular expression has a counterpart with the same language.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Conversion of Regular Expressions to Finite Automata and vice versa.

Simple explanation

Conversion lets us move between a pattern formula and a state machine while keeping the accepted language unchanged.

Formal / technical definition

Thompson construction maps an RE to an epsilon-NFA recursively. State elimination maps an automaton to an RE using labels that denote sets of paths.

Mental model

Each construction preserves the language, not necessarily the number of states or visual shape.

Notation to remember

For elimination, when removing state k, update each i-to-j label by R_ij | R_ik(R_kk)*R_kj; preserve start and final with a fresh entry and exit if useful.

Conversion of Regular Expressions to Finite Automata and vice versa.

How to work with this idea

- RE to automaton: create a two-state edge for each symbol; combine fragments with epsilon edges for union, concatenation, and star.
- Automaton to RE: add a unique start and final; label absent edges with empty-set.
- Eliminate one intermediate state at a time with the update formula.
- Simplify the resulting expression and verify with representative strings.

```
Eliminate k: direct i->j paths OR paths through k with any k-loop repetitions.
```

Solved example / construction

For a symbol a, make s -a-> f. For a|b, create a new entry that epsilon-branches to the a and b fragments, then epsilon-joins their exits. For a*, add a bypass for zero copies and a loop-back for repeated copies.

Conversion of Regular Expressions to Finite Automata and vice versa.

Common mistakes

- When eliminating k, include its self-loop as (R_kk)*.
- An absent edge means empty-set, not epsilon.
- Keep start/final states until the end; state order affects expression size, not correctness.

Exam / viva prompts

- Write the state-elimination recurrence.
- Why add fresh start/final states? It makes boundary handling uniform.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Each construction preserves the language, not necessarily the number of states or visual shape.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Arden’s Theorem.

Simple explanation

Arden's Theorem solves certain regular-expression equations, much like solving x = ax | b by gathering the repeated prefix into a star.

Mental model

Arden converts a recursive language equation into a closed regular expression.

Formal / technical definition

If X = AX union B for language variables and epsilon is not in A, the unique solution is X = A*B. Dually, X=XA union B has solution X=BA* under the analogous condition.

Notation to remember

The condition epsilon not in A matters for uniqueness. Language union is the equation's OR; concatenation is multiplication-like.

Arden’s Theorem.

How to work with this idea

- Write one equation per automaton state: R_i is the set of suffixes accepted starting from i.
- Collect transitions into concatenated language terms.
- Arrange each equation in X=AX union B or X=XA union B form.
- Apply Arden's Theorem and substitute until the start-state expression remains.

Solved example / construction

Solve X = aX union b. Here A={a}, B={b}; epsilon is not in {a}. Therefore X=a*b, meaning zero or more a symbols followed by b: b, ab, aab, ...

Arden’s Theorem.

Common mistakes

- Do not reverse the order in noncommutative concatenation: a*b differs from ba*.
- Check epsilon not in A before claiming uniqueness.
- The star applies to the coefficient A, not to the entire right-hand side.

Exam / viva prompts

- State the equation form, condition, and solution.
- Interpret a*b as all a-prefixes followed by b.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Arden converts a recursive language equation into a closed regular expression.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Concept of basic machine.

Simple explanation

A machine receives input, changes internal state, and produces a result. A recognizer answers yes/no; a transducer outputs a sequence of values.

Formal / technical definition

A basic machine can be modeled by states, an input alphabet, a transition rule, an initial configuration, and an output/acceptance rule. A finite-state machine restricts its control memory to finitely many states.

Mental model

A machine model is defined by its memory, transition rule, and observable result.

Notation to remember

Machine description separates the input, control state, transition behavior, and observation (accept/reject or output).

Concept of basic machine.

How to work with this idea

- Specify what counts as input and when input is read.
- Define the machine's configuration: at minimum its current state.
- Define each transition and any output produced.
- Define when the computation stops and how its result is interpreted.

Solved example / construction

A two-state parity machine consumes bits, updates E/O, and outputs the current parity after each bit. As a recognizer it accepts final E; as a transducer it can emit E/O after each step.

Concept of basic machine.

Common mistakes

- A recognizer and a transducer have different output conventions.
- A state describes control memory, not necessarily physical hardware.
- State clearly whether output is produced on transitions or in states.

Exam / viva prompts

- What components describe a machine? Input, configurations/states, transition rule, initial condition, and output or acceptance criterion.
- What is a transducer? A machine that maps input sequences to output sequences.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

A machine model is defined by its memory, transition rule, and observable result.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Properties and limitations of Finite State Machines (FSM).

Simple explanation

FSMs are simple, fast recognizers for properties that need only finite memory. They cannot remember an unbounded count or compare arbitrarily distant parts of an input.

Mental model

An FSM can remember a bounded summary, not an unbounded stack or tape.

Formal / technical definition

FSMs have finitely many configurations, so on a fixed input symbol their future behavior depends only on the current state. They recognize exactly regular languages; unbounded nested dependencies such as {a^n b^n:n>=0} are not regular.

Notation to remember

Finite memory does not mean finite input length. An FSM can process arbitrarily long inputs while cycling among finitely many states.

Properties and limitations of Finite State Machines (FSM).

How to work with this idea

- Ask what information about the prefix affects future acceptance.
- If a finite summary suffices, define it as states and build transitions.
- If arbitrary matching/counting is needed, test regularity with distinguishable prefixes or the pumping lemma.
- Choose a stronger model such as a PDA only when the property requires stack memory.

Solved example / construction

Parity is FSM-computable with 2 states. The language {a^n b^n:n>=0} is not regular: after an arbitrary number of a's, the machine would need distinct states to remember each n so it can match the later b-count.

Properties and limitations of Finite State Machines (FSM).

Common mistakes

- FSMs are not limited to short strings.
- A large finite counter has a fixed bound; it cannot count without bound.
- Do not claim a language is nonregular only because one chosen DFA failed.

Exam / viva prompts

- Give one strength and one limitation.
- Why is {a^n b^n} outside finite memory? Arbitrarily many prefix lengths require distinguishable future behavior.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

An FSM can remember a bounded summary, not an unbounded stack or tape.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Moore Machines.

Simple explanation

A Moore machine attaches its output to the current state. When the state changes, the reported output changes on the next state observation.

Mental model

Moore: output = lambda(state).

Formal / technical definition

A Moore machine is (Q,Sigma,Delta,q0,Gamma,lambda), with Delta:Q x Sigma->Q and lambda:Q->Gamma. Output depends only on state.

Notation to remember

For an input of length n, the state-output trace includes the initial state's output and then one output after each transition (often n+1 values).

Moore Machines.

How to work with this idea

- Choose states that encode the information needed for future behavior and output.
- Assign an output value to each state.
- Define a transition for every state/input pair.
- Trace initial output first, then the destination-state output after each symbol.

Solved example / construction

A parity Moore machine has states E and O with outputs 0 and 1. On input 101, outputs are 0 initially, then 1, 1, 0; the final parity is even.

Moore Machines.

Common mistakes

- Do not place output labels on edges when describing a Moore machine.
- Count the initial output when the convention requires it.
- Two states with different outputs cannot be merged in an equivalent Moore machine.

Exam / viva prompts

- What does lambda depend on? State only.
- What is a common timing difference from Mealy? Moore output is associated with a state and is observed after entering it.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Moore: output = lambda(state).

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Mealy Machines.

Simple explanation

A Mealy machine attaches output to each transition. The output can depend on both the current state and the input symbol just read.

Formal / technical definition

A Mealy machine is (Q,Sigma,Gamma,Delta,lambda,q0), with Delta:Q x Sigma->Q and lambda:Q x Sigma->Gamma. Output is produced per transition.

Mental model

Mealy: output = lambda(state,input).

Notation to remember

For an input of length n, a Mealy trace commonly has n outputs and no separate initial output.

Mealy Machines.

How to work with this idea

- Choose states summarizing the past.
- For each state and input, specify both next state and output.
- Trace each input symbol as a pair: destination/output.
- Check whether output timing is edge-based, especially for the first symbol.

Solved example / construction

A parity Mealy machine can output the new parity when reading each bit: from E on 1 go to O and output 1; from O on 1 go to E and output 0; on 0 remain and report the current parity.

Mealy Machines.

Common mistakes

- Do not assign a single output to a state in a Mealy diagram.
- Output is made on a transition; define the initial-output convention separately if needed.
- Transitions with the same destination may still emit different outputs.

Exam / viva prompts

- What does lambda depend on? Current state and current input.
- How many outputs for n input symbols? Usually n.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Mealy: output = lambda(state,input).

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Equivalence of Moore and Mealy Machines.

Simple explanation

Either machine can implement the same input-output behavior, though conversion may change the number of states and when the first output appears.

Formal / technical definition

A Moore machine can be converted to a Mealy machine by labeling each transition p -a-> q with the output of q. A Mealy machine can be converted to Moore form by splitting states as needed so each new state has one fixed output.

Mental model

Moore and Mealy differ in output placement; conversions preserve behavior after aligning timing.

Notation to remember

Equivalence must specify output alignment: initial Moore output versus transition outputs, and whether the initial Mealy output is defined.

Equivalence of Moore and Mealy Machines.

How to work with this idea

- For Moore-to-Mealy, copy destination-state output to each incoming edge.
- For Mealy-to-Moore, make states represent (original state, output just produced) when different incoming outputs require separation.
- Redirect transitions to the appropriate split states.
- Compare output sequences under the same timing convention.

Solved example / construction

If Moore state q has output 1, every incoming transition to q receives output 1 in the converted Mealy machine. If a Mealy state p is reached with outputs 0 and 1 on different edges, split p into two Moore states with those outputs.

Equivalence of Moore and Mealy Machines.

Common mistakes

- Conversion can require state splitting; a one-to-one mapping is not guaranteed.
- Ignoring initial output can create an apparent one-position mismatch.
- Preserve the output sequence, not just the state graph.

Exam / viva prompts

- Explain each conversion rule.
- Why might Mealy-to-Moore add states? A Moore state must have a single fixed output.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Moore and Mealy differ in output placement; conversions preserve behavior after aligning timing.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Module II: Regular Sets and Grammars

15 syllabus topics | definitions, constructions, proofs and review



Pumping Lemma for Regular Sets.

Simple explanation

A sufficiently long accepted string in a regular language must revisit a state. The loop between repeated visits can be repeated any number of times and the machine still accepts.

Formal / technical definition

If L is regular, there is p>=1 such that every s in L with |s|>=p can be written s=xyz with |xy|<=p, |y|>=1, and xy^i z in L for every i>=0.

Mental model

Use the pumping lemma to disprove regularity by defeating every permitted split.

Notation to remember

The lemma is a necessary property of regular languages. Its usual use is contradiction: assume regular, choose a witness, and show every legal split breaks for some i.

Pumping Lemma for Regular Sets.

How to work with this idea

- Assume L is regular and let p be its pumping length.
- Choose a string s in L with |s|>=p, selected to expose the claimed dependency.
- Consider an arbitrary split s=xyz satisfying the two bounds.
- Find an i (often 0 or 2) for which xy^i z is not in L; conclude L is not regular.

Solved example / construction

For L={a^n b^n:n>=0}, choose s=a^p b^p. Since |xy|<=p, y consists only of a's and has positive length. Pump i=0: fewer a's than b's, so the result is not in L. Contradiction.

Pumping Lemma for Regular Sets.

Common mistakes

- You choose s; you do not choose the decomposition.
- The proof must handle every legal split, not one convenient split.
- The lemma cannot generally prove regularity; a DFA/RE construction can.

Exam / viva prompts

- State all three conditions on x,y,z.
- Why must y be nonempty? Otherwise pumping changes nothing.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Use the pumping lemma to disprove regularity by defeating every permitted split.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Applications of Pumping Lemma.

Simple explanation

The pumping lemma is mainly a nonregularity tool. It reveals when a language needs more memory than a finite automaton can provide.

Mental model

A rigorous application is a quantified contradiction, not a pattern-based guess.

Formal / technical definition

A pumping proof uses the quantified implication: for every proposed p, choose s; for every legal decomposition, choose a pumping exponent i yielding a string outside L.

Notation to remember

The order of choices is crucial: adversarial split comes after the witness string, and pumping exponent comes after the split.

Applications of Pumping Lemma.

How to work with this idea

- Identify the unbounded relation: equal counts, mirrored halves, or a boundary-dependent condition.
- Choose a witness whose first p symbols force y into a controlled region.
- Classify possible y locations allowed by |xy|<=p.
- Pump down or up and show the defining property fails.

Solved example / construction

For L={0^n1^n:n>=0}, choose 0^p1^p; y is only 0s; pumping down breaks equal counts. For L={ww:w in {0,1}*}, choose s=(0^p1)(0^p1). Since |xy|<=p, y=0^k for k>0 lies in the first zero block. Pump down to 0^(p-k)1 0^p1. If k is odd the total length is odd; if k is even, the two equal-length halves place their first 1 at different offsets. Either way the result is not a square.

Applications of Pumping Lemma.

Common mistakes

- Do not state only 'counts become unequal' without proving y lies entirely in the chosen block.
- Check that the pumped string violates the language definition, not just its original form.
- Some nonregular languages need a different proof technique when pumping splits are hard to control.

Exam / viva prompts

- Which quantifier mistake is common? Picking a favorable split instead of considering all legal splits.
- Which i is often simplest? i=0 removes y; i=2 duplicates it.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

A rigorous application is a quantified contradiction, not a pattern-based guess.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Closure properties of Regular Sets.

Simple explanation

Regular languages stay regular after many standard operations. If two patterns can each be recognized with finite memory, a product machine can track both at once.

Mental model

Closure proofs are constructive: show the resulting machine or expression.

Formal / technical definition

Regular languages are closed under union, intersection, complement, difference, concatenation, Kleene star, reversal, and homomorphism; these follow from constructions on automata or expressions.

Notation to remember

For intersection, product states are pairs (p,q); for complement, first make a DFA complete, then swap final and nonfinal states.

Closure properties of Regular Sets.

How to work with this idea

- Name the operation and the automata for its operands.
- Choose the relevant construction: product for intersection; final-state swap for complement; epsilon-NFA join for union/concatenation/star.
- State the new start/final states and transitions.
- Explain why accepted inputs match the desired set operation.

```
(p,q) --a--> (delta_A(p,a), delta_B(q,a)); final iff p in F_A and q in F_B.
```

Solved example / construction

If A recognizes strings with an even number of a's and B recognizes strings ending in b, the product DFA state (p,q) tracks both facts. Mark a pair final only when p is even and q is 'ends in b'; it recognizes L(A) intersection L(B).

Closure properties of Regular Sets.

Common mistakes

- Complement requires a complete deterministic machine before swapping finals.
- Intersection product final states require both components final.
- Closure under an operation does not mean the original languages are equal.

Exam / viva prompts

- Give the product-state rule for intersection.
- How is complement formed from a complete DFA? Exchange accepting and rejecting states.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Closure proofs are constructive: show the resulting machine or expression.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Myhill-Nerode Theorem.

Simple explanation

Two prefixes are equivalent if no future suffix can make one accepted and the other rejected. A regular language needs only finitely many such future-behavior classes.

Formal / technical definition

For L subset Sigma*, define x ~_L y iff for every z in Sigma*, xz in L exactly when yz in L. L is regular iff ~_L has finitely many equivalence classes. The classes correspond to states of the minimal DFA.

Mental model

Myhill-Nerode gives both a regularity criterion and a minimum-state characterization.

Notation to remember

A distinguishing suffix z proves x and y inequivalent. The number of equivalence classes is the minimum DFA state count.

Myhill-Nerode Theorem.

How to work with this idea

- Choose candidate prefixes that may require different memory.
- For each pair, find a suffix z that makes exactly one concatenation belong to L.
- Use the resulting pairwise distinctions to prove a lower bound on states.
- For finite index, construct one state per class and define transitions by appending a symbol.

Solved example / construction

For L={w over {a} with |w| divisible by 3}, prefixes a^0, a^1, a^2 are pairwise distinguishable: choose suffix lengths 0, 2, or 1 as needed to make one total divisible by 3 and the other not. Exactly three residue classes give a 3-state minimal DFA.

Myhill-Nerode Theorem.

Common mistakes

- Equivalence tests all suffixes; one distinguishing suffix is enough to show non-equivalence.
- For regularity, finitely many classes are required.
- Distinguishable prefixes yield a state lower bound, not an upper bound.

Exam / viva prompts

- State the x~y definition.
- How do equivalence classes relate to a minimal DFA? One class per state.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Myhill-Nerode gives both a regularity criterion and a minimum-state characterization.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Minimization of Finite Automata.

Simple explanation

A DFA is minimal when no two states can be merged without changing which strings are accepted. Minimization preserves the language while reducing redundant memory states.

Formal / technical definition

Two DFA states p,q are equivalent iff every suffix leads either both to acceptance or both to rejection. A minimal DFA has one state per right-language equivalence class among reachable states.

Mental model

Minimization merges exactly the reachable states with identical future behavior.

Notation to remember

Only reachable states matter. The minimum DFA is unique up to renaming states.

Minimization of Finite Automata.

How to work with this idea

- Remove unreachable states from the start state.
- Partition states into accepting and non-accepting blocks.
- Refine blocks when a symbol sends members to different blocks.
- Stop when stable; each block becomes one state and transitions follow the original block destinations.

Solved example / construction

If p and q are both final, but on b one reaches a final block and the other a nonfinal block, they are distinguishable and must be split. A suffix beginning with b witnesses the difference.

Minimization of Finite Automata.

Common mistakes

- States with different acceptance status can never merge.
- Unreachable states should be removed before partition refinement.
- The goal is language preservation, not simply fewer states by visual inspection.

Exam / viva prompts

- What conditions allow merging? Same acceptance behavior for every suffix.
- Why minimal DFA unique only up to renaming? The equivalence classes are fixed but labels are arbitrary.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Minimization merges exactly the reachable states with identical future behavior.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Minimization Algorithms.

Simple explanation

Partition refinement repeatedly asks whether two states behave differently after some next symbol. If their destinations fall into different groups, the states must be separated.

Formal / technical definition

Table-filling marks pairs with different finality, then iteratively marks (p,q) if some a sends (delta(p,a),delta(q,a)) to an already marked pair. Unmarked pairs are equivalent. Partition refinement is the block-based counterpart.

Mental model

Table filling and partition refinement compute the same equivalence relation.

Notation to remember

For n states there are n(n-1)/2 unordered pairs. A marking witness can be recorded as a distinguishing suffix.

Minimization Algorithms.

How to work with this idea

- Table filling: mark final/nonfinal pairs first.
- For each unmarked pair and each symbol, inspect the destination pair.
- Mark if any destination pair is marked; repeat to a fixed point.
- Merge unmarked pairs, after removing unreachable states.

```
Pair p,q -> on a reaches (f,r); since f and r differ in finality, mark (p,q) with witness a.
```

Solved example / construction

Suppose p and q are both nonfinal. On a, p goes to final f while q goes to nonfinal r; then p,q are distinguished by suffix a and are marked immediately. If destinations are a marked pair after two steps, the witnessing suffix has that first symbol followed by the shorter witness.

Minimization Algorithms.

Common mistakes

- Treat a pair as unordered and avoid comparing a state with itself.
- Iterate until no new pair is marked.
- A pair left unmarked is equivalent only after the fixed point is reached.

Exam / viva prompts

- How does a witness string grow during table filling? Prefix the current symbol to the destination pair's witness.
- What is the initial marking rule? Pairs with one final and one nonfinal state.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Table filling and partition refinement compute the same equivalence relation.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Definition of Grammar.

Simple explanation

A grammar is a rule system for building strings. It starts from a designated symbol and repeatedly replaces nonterminal symbols until only terminals remain.

Formal / technical definition

A grammar is G=(V,T,P,S): V is a finite set of variables/nonterminals, T a terminal alphabet disjoint from V, P a finite set of productions, and S in V the start symbol. L(G)={w in T*: S =>* w}.

Mental model

A grammar defines a language through derivations from its start symbol.

Notation to remember

One derivation step applies one production to one occurrence of its left-hand side. =>* means zero or more steps; a sentential form may still contain nonterminals.

Definition of Grammar.

How to work with this idea

- Identify terminals (literal output symbols) and nonterminals (rewrite variables).
- Write each production as a left-hand pattern and right-hand replacement.
- Begin with S and apply rules until a terminal-only string is produced.
- The language is the set of all terminal strings derivable from S.

Solved example / construction

For S -> aSb | epsilon, derivations include S => epsilon and S => aSb => aabb. The generated language is {a^n b^n:n>=0}; each use adds one a to the left and one b to the right.

Definition of Grammar.

Common mistakes

- The empty production derives epsilon; it does not mean the empty language.
- A sentential form containing a nonterminal is not yet a generated word.
- The grammar tuple must specify its start symbol.

Exam / viva prompts

- What are the four parts of G? V,T,P,S.
- What does S=>*w mean? A finite sequence of zero or more production applications derives w.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

A grammar defines a language through derivations from its start symbol.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Regular Grammar.

Simple explanation

A regular grammar has productions shaped so that derivation can be followed by finite-state control. It generates exactly the regular languages.

Formal / technical definition

A right-linear grammar uses A -> aB, A -> a, and optionally S -> epsilon under suitable convention; a left-linear grammar uses A -> Ba, A -> a. Mixing left- and right-linear forms can exceed the regular-grammar definition.

Mental model

Regular grammars are grammar-shaped descriptions of finite-state behavior.

Notation to remember

Nonterminals correspond to states; a rule A->aB is a transition A -a-> B, and a terminal-only rule leads to a final state.

Regular Grammar.

How to work with this idea

- Check each production's shape.
- Choose right-linear or left-linear orientation consistently.
- For conversion to an automaton, make nonterminals states and route terminal-only rules to a fresh final state.
- For a DFA/NDFA-to-grammar conversion, create one production per transition and final-state epsilon rules.

Solved example / construction

A -> aB | b generates strings that start with a and then continue from B, or choose b and stop. In the automaton mapping, A -a-> B and A -b-> f, where f is final.

Regular Grammar.

Common mistakes

- A->aBC is not regular grammar form because two variables follow the terminal.
- Do not mix left-linear and right-linear productions casually.
- A regular grammar may be nondeterministic; it need not be a DFA itself.

Exam / viva prompts

- State right-linear rule forms.
- What language class does a regular grammar generate? Regular languages.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Regular grammars are grammar-shaped descriptions of finite-state behavior.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Context Free Grammar (CFG).

Simple explanation

A CFG rewrites one nonterminal at a time, no matter what symbols surround it. This freedom supports nested structures such as balanced parentheses.

Formal / technical definition

A CFG has productions A -> alpha, where A is one nonterminal and alpha is any string over terminals and nonterminals, including epsilon where allowed.

Mental model

CFG productions rewrite one nonterminal and can express recursive nesting.

Notation to remember

Derivation trees record the hierarchical choices. A language generated by some CFG is context-free.

Context Free Grammar (CFG).

How to work with this idea

- Choose a nonterminal for each recursive structure.
- Use alternatives for base cases and recursive cases.
- Derive a target string and draw its parse tree.
- Check that every production has exactly one nonterminal on its left.

Solved example / construction

S -> (S)S | epsilon generates balanced-parenthesis strings. To derive '()()': S => (S)S => ()S => ()(S)S => ()()S => ()(). Each recursive production adds a matched pair and allows a following balanced part.

Context Free Grammar (CFG).

Common mistakes

- The left side of every CFG production is one variable.
- A derivation sequence is not the same object as a parse tree.
- The grammar can be ambiguous even when every production is context-free.

Exam / viva prompts

- Why is the grammar context-free? A single variable is rewritten independently of its neighbors.
- Give a CFG example for balanced parentheses.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

CFG productions rewrite one nonterminal and can express recursive nesting.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Context Sensitive Grammar (CSG).

Simple explanation

A context-sensitive rule may rewrite a symbol only in a particular surrounding context and normally does not shorten the sentential form. This allows dependencies beyond a single stack.

Formal / technical definition

A standard noncontracting form uses productions alpha A beta -> alpha gamma beta with gamma nonempty; equivalently, productions do not decrease length, with a limited S->epsilon exception when S is not on any right side.

Mental model

Context sensitivity adds bounded contextual control while keeping derivations noncontracting.

Notation to remember

Context-sensitive languages are recognized by linear bounded automata. Details of the epsilon convention vary by textbook, so state the convention used.

Context Sensitive Grammar (CSG).

How to work with this idea

- Inspect whether the production's left side includes context around a variable.
- Check noncontraction: right-side length is at least left-side length, subject to the start exception.
- For membership intuition, track bounded but input-proportional working space.
- Compare the production form with CFG and unrestricted grammar forms.

Solved example / construction

The grammar for {a^n b^n c^n:n>=1} requires synchronizing three unbounded counts and is context-sensitive but not context-free. A schematic rule system must preserve or expand symbols; a PDA cannot store two independent unbounded count relations on one stack.

Context Sensitive Grammar (CSG).

Common mistakes

- A CSG production need not have a single variable on its left.
- Do not call every noncontracting grammar context-free.
- Mention the special epsilon/start-symbol convention when relevant.

Exam / viva prompts

- Which machine class recognizes context-sensitive languages? Linear bounded automata.
- What is the core restriction on productions? They are noncontracting under the standard convention.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Context sensitivity adds bounded contextual control while keeping derivations noncontracting.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Ambiguity in Grammars.

Simple explanation

A grammar is ambiguous if one generated string has two genuinely different structural explanations. The same word can have distinct parse trees or distinct leftmost derivations.

Formal / technical definition

A CFG is ambiguous if there exists w in L(G) with two distinct parse trees; equivalently, at least two distinct leftmost derivations for w.

Mental model

To prove ambiguity, exhibit one word with two parse trees.

Notation to remember

Operator precedence and associativity are common structural rules used to remove ambiguity from expression grammars.

Ambiguity in Grammars.

How to work with this idea

- Choose a candidate string with competing groupings.
- Construct two parse trees or leftmost derivations for that same string.
- Show the structures differ, not merely the order of independent rewrite steps.
- Refactor grammar with precedence levels or explicit associativity if an unambiguous grammar is required.

Solved example / construction

For E -> E+E | E*E | id, the string id+id*id has trees for id+(id*id) and (id+id)*id. To encode usual precedence, separate E (addition), T (multiplication), and F (atoms).

Ambiguity in Grammars.

Common mistakes

- Two derivation orders leading to the same tree do not prove ambiguity.
- Ambiguity belongs to a grammar; a language can have a different unambiguous grammar.
- Some context-free languages are inherently ambiguous, but a particular ambiguous grammar alone does not prove that.

Exam / viva prompts

- Define ambiguity using one string and two parse trees.
- How encode precedence? Use layered nonterminals, one level per precedence.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

To prove ambiguity, exhibit one word with two parse trees.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Reduced forms.

Simple explanation

A reduced grammar contains only symbols and rules that can contribute to some generated terminal string. Removing irrelevant machinery makes proofs and conversions easier.

Mental model

A reduced grammar preserves the language while excluding non-contributing symbols and rules.

Formal / technical definition

A reduced CFG is typically free of useless symbols and, depending on the target form, unit productions and unnecessary epsilon productions. Exact use of 'reduced form' varies; state which reductions are included.

Notation to remember

A symbol is useful when it is both generating (can derive terminals) and reachable from S. Both properties are required.

Reduced forms.

How to work with this idea

- Find generating variables by repeatedly marking A when A can derive a string of terminals and already marked variables.
- Remove nongenerating variables and productions containing them.
- Find variables reachable from S in the remaining grammar.
- Remove unreachable variables; then apply unit/epsilon elimination required by the target normal form.

Solved example / construction

If S -> AB, A -> a, and B -> B, then B is nongenerating: it never yields terminals. Remove B and S->AB; if no useful derivation remains from S, the language is empty. By contrast, a generating variable never reachable from S is also removable.

Reduced forms.

Common mistakes

- Generating and reachable are separate tests.
- Do not remove a symbol merely because it does not appear in one chosen derivation.
- After a reduction stage, recompute reachability on the remaining grammar.

Exam / viva prompts

- What makes a variable useful? It is reachable and generating.
- Why remove useless symbols before conversion? They cannot contribute to terminal strings and complicate the grammar.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

A reduced grammar preserves the language while excluding non-contributing symbols and rules.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Removal of useless symbols and unit productions.

Simple explanation

Useless variables never help produce a terminal word from the start symbol. A unit production only renames one variable as another; it can be bypassed.

Formal / technical definition

A unit production has form A->B with A,B variables. Build the unit-pair relation A=>*_unit B; copy each non-unit production of B to A, then delete unit rules. Remove nongenerating and unreachable variables using the two-phase procedure.

Mental model

Eliminate unit chains by copying reachable non-unit choices, then delete the chains.

Notation to remember

Order matters: eliminate nongenerating symbols, then unreachable symbols, then unit productions; preserve epsilon behavior only if the target form permits it.

Removal of useless symbols and unit productions.

How to work with this idea

- Compute generating variables to a fixed point and remove rules involving nongenerating variables.
- Compute variables reachable from S and remove the rest.
- For each A, find every B reachable through zero or more unit rules.
- Copy B's non-unit productions to A and delete all unit productions.

Solved example / construction

Given A->B, B->C|b, C->c, unit closure from A includes A,B,C, so A receives A->b and A->c. The unit rules A->B and B->C are removed; generated terminal strings stay the same.

Removal of useless symbols and unit productions.

Common mistakes

- Do not copy a unit rule as if it were a terminal production.
- Take transitive closure: a chain A->B->C must be handled.
- Check cycles such as A->B, B->A; closure still terminates because the variable set is finite.

Exam / viva prompts

- What is a unit production? A->B for variables A and B.
- What must be preserved? The language generated from S.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Eliminate unit chains by copying reachable non-unit choices, then delete the chains.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Chomsky Normal Form (CNF).

Simple explanation

CNF standardizes CFG rules into either two variables or one terminal. That makes parse structure uniform and supports formal parsing algorithms.

Formal / technical definition

A CFG is in CNF if every production is A->BC or A->a, with optional S->epsilon when epsilon is in the language and S does not occur on any right side.

Mental model

CNF uses A->BC or A->a, plus the controlled start epsilon exception.

Notation to remember

Typical conversion: remove epsilon productions, unit productions, and useless symbols; replace terminals in long right sides; binarize long variable sequences.

Chomsky Normal Form (CNF).

How to work with this idea

- Introduce a fresh start variable if the old start appears on a right side or epsilon handling requires it.
- Remove epsilon productions while preserving permitted nullable combinations.
- Remove unit productions and useless symbols.
- Replace terminals inside long rules with helper variables; split right sides longer than two variables.

```
Before: A->aBCD. After: X_a->a, A->X_a D, D->BC (then order as needed).
```

Solved example / construction

Convert A->aBC to X_a->a and A->X_a B C, then binarize as A->D C and D->X_a B. The resulting rules have at most two variables on the right; each terminal appears alone.

Chomsky Normal Form (CNF).

Common mistakes

- A->BC allows two variables, not two terminals.
- A->a is allowed; A->epsilon only under the start exception.
- Track nullable variables carefully so the language is preserved.

Exam / viva prompts

- List the two standard production forms.
- Why is a fresh start variable sometimes introduced? To isolate epsilon and prevent start-symbol occurrences on right sides.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

CNF uses A->BC or A->a, plus the controlled start epsilon exception.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Greibach Normal Form (GNF).

Simple explanation

GNF forces every nonempty production to begin with a terminal. A derivation therefore emits one terminal at every step, making derivation length track output length.

Mental model

GNF begins each production with exactly one terminal, followed only by variables.

Formal / technical definition

A CFG is in GNF if productions have form A->aA1A2...Ak for k>=0, with a terminal a first and zero or more variables after it; S->epsilon may be allowed under the usual start exception.

Notation to remember

GNF conversion removes epsilon/unit/useless rules, orders variables, eliminates left recursion, and substitutes until each right side begins with a terminal.

Greibach Normal Form (GNF).

How to work with this idea

- Reduce the CFG and handle epsilon separately.
- Order variables so substitutions move toward variables whose leading form is known.
- Remove left recursion that prevents terminal-first rules.
- Substitute leading variables until every nonempty rule starts with a terminal.

Solved example / construction

A->BC and B->aD can be substituted to A->aDC, now terminal-first. If B also has multiple alternatives, substitute each one. Continue until every production has a terminal as its first symbol.

Greibach Normal Form (GNF).

Common mistakes

- GNF permits variables after the first terminal.
- CNF and GNF have different shapes; do not use A->BC as a GNF rule.
- Epsilon needs explicit handling for the start symbol.

Exam / viva prompts

- State the GNF rule form.
- What useful fact follows? A derivation of a nonempty word of length n uses n production steps in the standard leftmost derivation.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

GNF begins each production with exactly one terminal, followed only by variables.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

MODULE 3

Module III: Pushdown Automata and Turing Machines

15 syllabus topics | definitions, constructions, proofs and review

Introduction to Pushdown Machines.

Simple explanation

A pushdown machine is a finite-state controller with a stack. The stack provides last-in, first-out memory that can grow with the input, which is enough to track nested structure.

Formal / technical definition

A PDA typically has finite states, input alphabet, stack alphabet, transition relation, start state, initial stack symbol, and accepting states or an empty-stack acceptance rule.

Mental model

PDA = finite control plus one stack; stack actions model nested matching.

Notation to remember

A transition may read an input symbol or epsilon, inspect/pop the stack top, and replace it with a stack string. The top-of-stack convention must be stated.

Introduction to Pushdown Machines.

How to work with this idea

- Identify what must be remembered in last-in/first-out order.
- Choose stack symbols for the stored obligations.
- Push while creating obligations and pop when matching them.
- Use states to mark phases, such as before and after a separator.

```
Input: aaabb; stack: Z -> AZ -> AAZ -> AAAZ -> AAZ -> AZ -> Z; all input consumed.
```

Solved example / construction

For {a^n b^n:n>=0}, push one marker for each a. When b's begin, pop one marker per b. Accept when input ends exactly as the stack returns to its bottom marker. The stack stores an unbounded count in unary.

Introduction to Pushdown Machines.

Common mistakes

- A PDA stack is not a queue; only the top is directly accessible.
- A PDA cannot freely inspect the entire stack.
- Define whether the top is written at the left or right of stack strings.

Exam / viva prompts

- What capability does the stack add? Unbounded LIFO memory.
- Which language class is recognized by PDAs? Context-free languages, with standard nondeterministic PDA acceptance.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

PDA = finite control plus one stack; stack actions model nested matching.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Acceptance of PDA.

Simple explanation

A PDA can accept by reaching a designated final state or by emptying its stack, depending on the formal convention. For nondeterministic PDA, one accepting computation path is enough.

Mental model

Check state, unread input, stack, and acceptance convention together.

Formal / technical definition

A configuration is (q,w,gamma): current state, unread input, stack content. Acceptance by final state requires a reachable configuration (q_f,epsilon,gamma); empty-stack acceptance requires (q,epsilon,epsilon), with state unrestricted.

Notation to remember

For NPDAs, acceptance is existential over computation paths. Final-state and empty-stack modes are equivalent in language power, though conversion changes the machine.

Acceptance of PDA.

How to work with this idea

- Write the initial configuration, including initial stack symbol.
- Apply one transition at a time, updating state, unread input, and stack.
- Branch when several moves are available.
- After consuming all input, check the chosen acceptance condition exactly.

```
Configuration trace: (q0,aaabb,Z) -> (q1,aabb,AZ) -> ... -> (q1,epsilon,Z) -> accept by empty stack.
```

Solved example / construction

In the a^n b^n machine, after reading aaabb, the stack can be Z again with input empty. If using a final-state design, take an epsilon transition from the bottom-marker state to a final state. Record which convention your diagram uses.

Acceptance of PDA.

Common mistakes

- Reaching a final state before the input ends is not acceptance.
- Empty stack with unread input is not acceptance.
- Do not mix acceptance by empty stack with acceptance by final state in a trace.

Exam / viva prompts

- Define a PDA configuration.
- Do final-state and empty-stack acceptance have equal expressive power? Yes for nondeterministic PDAs, via conversion.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Check state, unread input, stack, and acceptance convention together.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Conversion of PDA to CFG and CFG to PDA.

Simple explanation

A CFG and a PDA describe the same context-free language from two sides: a grammar builds nested structures; a PDA checks them using a stack.

Mental model

CFG and PDA are equivalent models of context-free languages; conversions encode derivation or stack behavior.

Formal / technical definition

A CFG-to-PDA construction simulates a leftmost derivation: keep pending grammar symbols on the stack, expand a variable nondeterministically, and match terminals against input. A PDA-to-CFG construction uses variables describing computations that move between states while removing a stack symbol.

Notation to remember

A common PDA-to-CFG variable [p A q] represents strings that take the PDA from state p with A on top to state q after A has been removed, without disturbing the stack below A.

Conversion of PDA to CFG and CFG to PDA.

How to work with this idea

- CFG to PDA: push S; if top is a variable, replace it using a production; if top is terminal, match and pop it with the next input symbol.
- PDA to CFG: first normalize acceptance/stack behavior if required.
- Create variables representing state-to-state stack-removal behavior.
- Add productions mirroring each PDA transition and its possible continuation states.

Solved example / construction

For S->aSb | epsilon, the PDA can push a marker when it reads each leading a, switch phase at the midpoint nondeterministically, then read one b while popping each marker. This illustrates CFG-to-PDA correspondence; the reverse construction encodes each balanced stack episode with [p A q]-style variables.

Conversion of PDA to CFG and CFG to PDA.

Common mistakes

- A PDA-to-CFG conversion can create many variables and productions; define their meaning before listing rules.
- The stack bottom and acceptance mode affect the construction.
- Do not claim direct symbol-for-symbol equality between grammar variables and PDA states.

Exam / viva prompts

- What language class equality does the conversion establish? CFG languages = languages accepted by NPDAs.
- Explain [pAq] informally: strings that remove A while moving p to q.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

CFG and PDA are equivalent models of context-free languages; conversions encode derivation or stack behavior.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Applications of Pushdown Machines.

Simple explanation

A PDA is useful whenever input has nested, properly matched structure. The stack stores an obligation when an opening construct appears and checks it when the matching close appears.

Mental model

Pushdown machines are natural recognizers for nested and recursive syntax.

Formal / technical definition

PDAs recognize context-free languages. Typical applications include balanced delimiters, nested scopes, expression syntax, recursive constructs, and parsing, where one unbounded stack captures nested dependencies.

Notation to remember

A single stack naturally handles one nested dependency. It does not in general recognize arbitrary crossing dependencies requiring multiple independent stacks.

Applications of Pushdown Machines.

How to work with this idea

- Find the opening event and choose what to push.
- Find the corresponding closing event and pop/check the expected symbol.
- Use finite states to track phases and separators.
- Test empty, shallow, deeply nested, and mismatched cases.

Solved example / construction

For balanced parentheses, push a marker on '(' and pop on ')'. Reject if ')' arrives with no marker; accept at end only if no markers remain. Trace '(()())': stack heights 1,2,1,2,1,0, so it is balanced.

Applications of Pushdown Machines.

Common mistakes

- Checking only equal counts is insufficient: ')((' has equal counts but invalid order.
- A stack must preserve nesting order, not only count.
- An application example should state its input alphabet and acceptance condition.

Exam / viva prompts

- Why does a stack fit nested syntax? The most recently opened item must be closed first.
- How detect a mismatch? Compare each closing symbol to the top obligation.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Pushdown machines are natural recognizers for nested and recursive syntax.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Deterministic Turing Machines.

Simple explanation

A deterministic Turing machine is a precise abstract computer with a tape, a read/write head, and finite control. It can revisit and rewrite its work area.

Mental model

A DTM performs one uniquely determined sequence of tape configurations.

Formal / technical definition

A DTM transition maps (state, scanned tape symbol) to (next state, symbol written, head direction). At most one transition is available for each state-symbol pair; halting states or missing transitions stop computation.

Notation to remember

A configuration records state, tape contents, and head location. The tape is conceptually unbounded; only finitely many cells have been visited at any finite time.

Deterministic Turing Machines.

How to work with this idea

- Specify input encoding and blank symbol.
- Choose states for phases of the algorithm.
- For each phase, define what symbol is read, what is written, and which way the head moves.
- Provide accept, reject, and halting behavior; test short inputs and edge cases.

```
(q, scanned=1) -> (q, write=1, move=R); transition arrow label reads/writes,direction.
```

Solved example / construction

A machine can test whether a unary string has at least one symbol by reading the first cell: on 1, move to accept; on blank, reject. More complex machines mark symbols and sweep right/left to compare or transform parts of input.

Deterministic Turing Machines.

Common mistakes

- A TM can write as well as read; an FA cannot.
- A transition must specify state, written symbol, and head movement.
- A computation may fail to halt; it is not automatically a reject.

Exam / viva prompts

- What is a configuration? State, tape, and head position.
- What distinguishes a DTM? At most one next move per configuration.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

A DTM performs one uniquely determined sequence of tape configurations.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Non-Deterministic Turing Machines.

Simple explanation

An NTM can have several possible next moves. It accepts if at least one branch eventually accepts; branches can be pictured as a computation tree.

Mental model

DTM and NTM have equal computability power; nondeterminism changes branching, not which languages are computable.

Formal / technical definition

An NTM has a transition relation from state-symbol pairs to a set of actions. Its language contains inputs with at least one finite accepting computation branch.

Notation to remember

For language recognition, an accepting branch establishes membership. If no branch accepts, an NTM may reject by all branches halting or may run forever on some branches, depending on the machine definition.

Non-Deterministic Turing Machines.

How to work with this idea

- Draw the initial configuration as the root of a computation tree.
- Expand one child for each allowed move.
- Label leaves accept, reject, or nonhalting.
- For an input, accept when an accepting branch exists.

Solved example / construction

For a search problem, an NTM may guess a candidate certificate and deterministically verify it. The input is accepted if some guessed candidate passes verification.

Non-Deterministic Turing Machines.

Common mistakes

- Nondeterminism is not simultaneous physical execution in the definition.
- One accepting branch suffices; all branches need not accept.
- For decidability, remember the halting behavior of every relevant branch.

Exam / viva prompts

- How does NTM acceptance differ from DTM? Existential path versus one unique path.
- Do deterministic and nondeterministic TMs recognize the same class of languages? Yes; they may differ in efficiency.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

DTM and NTM have equal computability power; nondeterminism changes branching, not which languages are computable.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Design of Turing Machines.

Simple explanation

Designing a TM means converting an informal algorithm into explicit tape actions. Reliable designs use phases, markers, and invariants so each sweep has a clear purpose.

Mental model

A good TM design is an algorithm plus explicit tape conventions and a correctness argument.

Formal / technical definition

A TM design is correct if every transition sequence implements the stated algorithm, halts as required, and accepts exactly the target language. A high-level description can be refined into a transition table.

Notation to remember

An invariant states what remains true at a phase boundary, such as 'all symbols left of the head are matched pairs'.

Design of Turing Machines.

How to work with this idea

- State the language and input encoding precisely.
- Describe a high-level algorithm in phases and choose markers.
- For each sweep, state the head's stopping condition and invariant.
- Handle empty/short inputs and malformed symbols.
- Refine to transitions and prove soundness, completeness, and halting if required.

Solved example / construction

To recognize {a^n b^n:n>=1}, repeatedly mark the leftmost unmarked a as X, scan right to the leftmost unmarked b and mark Y, return left, and repeat. Reject if a required b is absent or symbols are out of order; accept when no unmarked a remains and no unmarked b remains.

Design of Turing Machines.

Common mistakes

- A vague instruction like 'compare the halves' is not yet a machine design.
- Every scan needs a stopping condition.
- Prove both directions: valid inputs accept; invalid inputs cannot accept.

Exam / viva prompts

- What makes a TM description rigorous? Explicit phases, transition actions, and correctness invariants.
- What must a complete recognizer specify? How it handles malformed and boundary inputs.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

A good TM design is an algorithm plus explicit tape conventions and a correctness argument.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Halting Problem.

Simple explanation

There is no single algorithm that can correctly decide, for every program and input, whether that computation will eventually stop.

Formal / technical definition

The halting problem H={(M,w): TM M halts on input w} is undecidable. A diagonal argument assumes a decider and constructs a machine that contradicts its prediction on its own encoding.

Mental model

The Halting Problem is undecidable because self-reference defeats any proposed universal decider.

Notation to remember

Undecidable does not mean every instance is impossible to resolve; many particular programs can be analyzed. It means no total algorithm solves all encoded instances.

Halting Problem.

How to work with this idea

- Assume a decider HALT(M,w) exists.
- Build D(x): if HALT(x,x) says halts, loop forever; otherwise halt.
- Run D on its own encoding d.
- Either answer contradicts D's behavior, so the assumed decider cannot exist.

Solved example / construction

If HALT(d,d)=halts, D(d) is defined to loop, contradiction. If HALT(d,d)=does-not-halt, D(d) halts, contradiction. Therefore H has no total decider.

Halting Problem.

Common mistakes

- A recognizer for halting can simulate M and accept if it halts; it may run forever otherwise. That does not decide H.
- Undecidable is not the same as unrecognizable.
- The proof concerns all machine-input pairs, not one program.

Exam / viva prompts

- What contradiction is used? A machine that does the opposite of the predicted behavior on its own code.
- Can the halting set be recognized? Yes, by simulation; it is not decidable.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

The Halting Problem is undecidable because self-reference defeats any proposed universal decider.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Post Correspondence Problem (PCP).

Simple explanation

PCP asks whether a sequence of domino-like string pairs can be arranged, with repetition allowed, so the top and bottom concatenations match exactly.

Formal / technical definition

An instance is a finite set of pairs (u_i,v_i). The question is whether there is a nonempty index sequence i1...ik such that u_i1...u_ik = v_i1...v_ik. General PCP is undecidable.

Mental model

PCP is a string-concatenation matching problem; the general decision problem is undecidable.

Notation to remember

The same tile may be used more than once. The order is shared by top and bottom; equality is tested only after concatenating the full sequence.

Post Correspondence Problem (PCP).

How to work with this idea

- List the available tiles and candidate index sequence.
- Concatenate top strings in that order.
- Concatenate bottom strings in the same order.
- Compare complete strings; for theory, distinguish an exhibited solution from a proof that none exists.

Solved example / construction

Tiles (a,ab) and (ba,a) have sequence 1,2: top=a·ba=aba; bottom=ab·a=aba. Hence this PCP instance has a solution. A finite failed search cannot establish no solution for arbitrary PCP.

Post Correspondence Problem (PCP).

Common mistakes

- Do not compare tile-by-tile; only total concatenations must match.
- A solution is a nonempty sequence of indices.
- Finding one sequence proves solvability; proving unsolvability is generally not a finite brute-force task.

Exam / viva prompts

- State the PCP equality condition.
- Why is PCP important? It is a classic undecidable problem used in reductions.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

PCP is a string-concatenation matching problem; the general decision problem is undecidable.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Chomsky Hierarchy of Grammars.

Simple explanation

The hierarchy organizes grammars and languages by how restrictive their production rules are. Less restrictive rules can describe more languages.

Formal / technical definition

Type 3 regular grammars are included in Type 2 context-free grammars, included in Type 1 context-sensitive grammars, included in Type 0 unrestricted grammars. The corresponding machine families are finite automata, PDAs, linear bounded automata, and Turing machines.

Mental model

More permissive grammar forms correspond to stronger machine models.

Notation to remember

Containment is strict at each level in the standard hierarchy. A language at a higher level may still happen to belong to a lower one.

Chomsky Hierarchy of Grammars.

How to work with this idea

- Classify a grammar by its production restrictions.
- Associate the language family and recognizing machine.
- Use inclusion to infer that every lower-class language also belongs to each higher class.
- Use known counterexamples to show an inclusion is strict.

```
Type 3 FA -> Type 2 PDA -> Type 1 LBA -> Type 0 TM.
```

Solved example / construction

Regular subset Context-Free subset Context-Sensitive subset Recursively Enumerable. Examples separating adjacent levels include {a^n b^n} (context-free but not regular) and {a^n b^n c^n} (context-sensitive but not context-free).

Chomsky Hierarchy of Grammars.

Common mistakes

- The hierarchy is nested, not a set of disjoint categories.
- A language can be generated by grammars of different types; classify by the least expressive class that contains it.
- Do not confuse context-sensitive languages with recursively enumerable languages.

Exam / viva prompts

- Map each grammar type to its machine.
- Which inclusion is the finite-automata/PDA boundary? Regular is properly contained in context-free.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

More permissive grammar forms correspond to stronger machine models.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Unrestricted Grammars.

Simple explanation

An unrestricted grammar places almost no shape restriction on production rules. It can describe exactly the recursively enumerable languages.

Formal / technical definition

A Type 0 production has form alpha -> beta, where alpha contains at least one nonterminal and alpha,beta are strings over variables and terminals; no noncontracting restriction is imposed.

Mental model

Unrestricted grammars capture Turing-recognizable languages through unbounded derivation search.

Notation to remember

Unrestricted derivations can erase symbols or increase length. A Turing machine can simulate derivation search and accept when a target word is generated.

Unrestricted Grammars.

How to work with this idea

- Check whether the left side contains at least one nonterminal.
- Do not require length preservation or a single variable on the left.
- Interpret derivations as an unbounded search over possible rewrites.
- Relate generated strings to Turing-machine recognition.

Solved example / construction

To recognize membership in L(G), enumerate derivations breadth-first and accept if the target word appears. If it is absent, the search may continue forever, which matches recognizability rather than decidability.

Unrestricted Grammars.

Common mistakes

- A derivation search that fails to find w does not prove w is absent.
- A production with terminal-only left side is not a standard useful Type 0 production.
- Unrestricted is the most general class in the hierarchy, not a claim that all problems are decidable.

Exam / viva prompts

- Which class corresponds to Type 0 grammars? Recursively enumerable languages / Turing-recognizable languages.
- Why may membership search not halt on nonmembers? Derivations can be unbounded.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Unrestricted grammars capture Turing-recognizable languages through unbounded derivation search.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Context Sensitive Languages.

Simple explanation

A context-sensitive language needs more memory than one stack can provide, but its membership can be checked using space proportional to the input length.

Formal / technical definition

A language is context-sensitive iff generated by a Type 1 grammar under the standard conventions, equivalently accepted by a nondeterministic linear bounded automaton (LBA), whose tape is bounded by a linear function of input length.

Mental model

Context-sensitive computation is Turing-like but uses only input-proportional tape.

Notation to remember

LBA tape is input-proportional, unlike the unrestricted unbounded work tape of a general TM.

Context Sensitive Languages.

How to work with this idea

- Identify a dependency that a single stack cannot store.
- Consider whether input-length-bounded workspace suffices.
- Classify the grammar as noncontracting or the recognizer as an LBA.
- Keep the relationship to context-free and recursively enumerable classes explicit.

Solved example / construction

The language {a^n b^n c^n:n>=1} is a standard context-sensitive example. Its three counts must agree; an LBA can mark and revisit input-proportional cells to coordinate them, whereas one PDA stack cannot compare all three counts.

Context Sensitive Languages.

Common mistakes

- An LBA is not an ordinary DFA or PDA.
- Linear bounded refers to tape space, not runtime.
- Context-sensitive does not mean that rules can depend on arbitrary natural-language context; use the formal production definition.

Exam / viva prompts

- What is the machine model for CSLs? Linear bounded automaton.
- What resource is bounded? Work tape length by O(input length).

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Context-sensitive computation is Turing-like but uses only input-proportional tape.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Relationship among language classes.

Simple explanation

Language classes form a ladder of increasing expressive power. A stronger model can simulate weaker ones, so each lower class is contained in the next.

Formal / technical definition

Regular subset CFL subset CSL subset RE. Regular languages are recognized by FA; CFLs by NPDAs; CSLs by LBAs; recursively enumerable languages by TMs. Decidable languages are a proper subset of RE and contain the context-sensitive languages.

Mental model

Classify by expressive power and always distinguish acceptance from guaranteed termination.

Notation to remember

A proper subset means inclusion plus at least one separating language. Membership, recognition, and decision are distinct: a recognizer may run forever on nonmembers; a decider halts on every input.

Relationship among language classes.

How to work with this idea

- Draw the nested classes before answering a comparison question.
- Name the canonical machine for each class.
- Use a separator example to show strictness.
- Distinguish recognition from decision when discussing computability.

```
Regular (FA) subset CFL (PDA) subset CSL (LBA) subset RE (TM); decidable subset RE.
```

Solved example / construction

A^nB^n is context-free but not regular; A^nB^nC^n is context-sensitive but not context-free. Every decidable language is recursively enumerable, but some RE languages such as the Halting Problem are not decidable.

Relationship among language classes.

Common mistakes

- Do not draw hierarchy levels as disjoint sets.
- A language can belong to multiple higher classes.
- RE is not the same as decidable: nonmembers may cause nontermination.

Exam / viva prompts

- Recite the inclusion chain and machines.
- What separates decidable from recognizable? A decider halts on every input.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Classify by expressive power and always distinguish acceptance from guaranteed termination.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Computability concepts.

Simple explanation

Computability asks whether any algorithm can solve a problem for every valid input. Complexity asks how many resources a solvable algorithm needs.

Mental model

Computability concerns whether a total effective solution exists; complexity concerns its resource cost.

Formal / technical definition

A decision problem is decidable if a TM halts on every input and accepts exactly yes-instances. It is recognizable if a TM accepts yes-instances but may loop on no-instances. A function is computable if an effective procedure returns its value for every input in its domain.

Notation to remember

Undecidability proofs commonly use diagonalization or mapping reductions. A reduction transfers a hypothetical solver for one problem into a solver for another.

Computability concepts.

How to work with this idea

- Translate the question into a language of yes-instances.
- Specify whether the machine must halt on all inputs or only accept members.
- For an undecidability proof, assume a solver and derive a contradiction or reduce a known undecidable problem.
- Separate existence of an algorithm from efficiency of an algorithm.

Solved example / construction

A program that simulates M on w and accepts when M halts recognizes the halting set, but it does not decide it: on nonhalting pairs the simulation never ends.

Computability concepts.

Common mistakes

- Undecidable does not mean computationally expensive; it means no total decider exists.
- Recognizable does not imply decidable.
- A reduction direction must be stated correctly: solve target to solve source, or vice versa, as the proof requires.

Exam / viva prompts

- Define decidable and recognizable.
- Can a recognizable problem have nonmembers on which the machine loops? Yes.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Computability concerns whether a total effective solution exists; complexity concerns its resource cost.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

Primitive Recursive Functions.

Simple explanation

Primitive recursive functions are built from simple starting functions using composition and recursion over a natural-number argument. Their definitions always finish after a bounded, structurally specified number of steps.

Formal / technical definition

The initial functions are zero, successor, and projections. The class is closed under composition and primitive recursion: f(x,0)=g(x), f(x,n+1)=h(x,n,f(x,n)). Every primitive recursive function is total and computable.

Mental model

Primitive recursion guarantees termination through a fixed natural-number iteration bound.

Notation to remember

Addition and multiplication are primitive recursive; Ackermann's function is computable but not primitive recursive, showing the class is not all total computable functions.

Primitive Recursive Functions.

How to work with this idea

- Start with zero, successor, or projection functions.
- Build new functions by composition of already defined functions.
- For recursion, give a base clause and a step clause using the previous value.
- Check totality: each value is obtained by finite induction on a natural number.

Solved example / construction

Addition: add(x,0)=x; add(x,n+1)=successor(add(x,n)). This uses primitive recursion over n and successor, so addition is primitive recursive. Multiplication can be repeated addition.

Primitive Recursive Functions.

Common mistakes

- Primitive recursion is not arbitrary recursion; the recursive argument decreases through a natural-number bound.
- Primitive recursive functions are total, but not every total computable function is primitive recursive.
- Do not confuse primitive recursive functions with primitive functions in calculus.

Exam / viva prompts

- Name the three initial function schemes.
- Are all primitive recursive functions total? Yes.
- What famous function is computable but not primitive recursive? Ackermann's function.

VIVA CHECK

Define the model, state its key rule, then walk through one input or derivation.

Key takeaway

Primitive recursion guarantees termination through a fixed natural-number iteration bound.

FAST SELF-CHECK

Can you state the definition, trace the example, and explain one failure case without looking?

One-line definitions to memorize

Finite Automata

- Finite State Systems: A finite-state system has a limited set of modes.
- Basic definitions: An alphabet is a set of symbols.
- Deterministic Finite Automata (DFA): A DFA never has to guess.
- Non-Deterministic Finite Automata (NDFA): An NDFA may have several possible next states for a state-symbol pair, or none.
- Equivalence of DFA and NDFA: A DFA and an NDFA can look different internally yet accept exactly the same strings.
- Finite Automata with ε-moves: An epsilon move changes state without consuming an input symbol.

Regular Expressions

- Regular expressions: A regular expression is a compact pattern description.
- Equivalence of Finite Automata and Regular Expressions: Finite automata and regular expressions are two descriptions of the same regular patterns: one is operational, the other algebraic.
- Conversion of Regular Expressions to Finite Automata and vice versa: Conversion lets us move between a pattern formula and a state machine while keeping the accepted language unchanged.
- Arden’s Theorem: Arden's Theorem solves certain regular-expression equations, much like solving x = ax | b by gathering the repeated prefix into a star.

Introduction to Machines

- Concept of basic machine: A machine receives input, changes internal state, and produces a result.
- Properties and limitations of Finite State Machines (FSM): FSMs are simple, fast recognizers for properties that need only finite memory.
- Moore Machines: A Moore machine attaches its output to the current state.
- Mealy Machines: A Mealy machine attaches output to each transition.
- Equivalence of Moore and Mealy Machines: Either machine can implement the same input-output behavior, though conversion may change the number of states and when the first output appears.

One-line definitions to memorize

Properties of Regular Sets

- Pumping Lemma for Regular Sets: A sufficiently long accepted string in a regular language must revisit a state.
- Applications of Pumping Lemma: The pumping lemma is mainly a nonregularity tool.
- Closure properties of Regular Sets: Regular languages stay regular after many standard operations.
- Myhill-Nerode Theorem: Two prefixes are equivalent if no future suffix can make one accepted and the other rejected.
- Minimization of Finite Automata: A DFA is minimal when no two states can be merged without changing which strings are accepted.
- Minimization Algorithms: Partition refinement repeatedly asks whether two states behave differently after some next symbol.

Grammars

- Definition of Grammar: A grammar is a rule system for building strings.
- Regular Grammar: A regular grammar has productions shaped so that derivation can be followed by finite-state control.
- Context Free Grammar (CFG): A CFG rewrites one nonterminal at a time, no matter what symbols surround it.
- Context Sensitive Grammar (CSG): A context-sensitive rule may rewrite a symbol only in a particular surrounding context and normally does not shorten the sentential form.
- Ambiguity in Grammars: A grammar is ambiguous if one generated string has two genuinely different structural explanations.
- Reduced forms: A reduced grammar contains only symbols and rules that can contribute to some generated terminal string.
- Removal of useless symbols and unit productions: Useless variables never help produce a terminal word from the start symbol.
- Chomsky Normal Form (CNF): CNF standardizes CFG rules into either two variables or one terminal.
- Greibach Normal Form (GNF): GNF forces every nonempty production to begin with a terminal.

One-line definitions to memorize

Pushdown Automata

- Introduction to Pushdown Machines: A pushdown machine is a finite-state controller with a stack.
- Acceptance of PDA: A PDA can accept by reaching a designated final state or by emptying its stack, depending on the formal convention.
- Conversion of PDA to CFG and CFG to PDA: A CFG and a PDA describe the same context-free language from two sides: a grammar builds nested structures; a PDA checks them using a stack.
- Applications of Pushdown Machines: A PDA is useful whenever input has nested, properly matched structure.

Turing Machines

- Deterministic Turing Machines: A deterministic Turing machine is a precise abstract computer with a tape, a read/write head, and finite control.
- Non-Deterministic Turing Machines: An NTM can have several possible next moves.
- Design of Turing Machines: Designing a TM means converting an informal algorithm into explicit tape actions.
- Halting Problem: There is no single algorithm that can correctly decide, for every program and input, whether that computation will eventually stop.
- Post Correspondence Problem (PCP): PCP asks whether a sequence of domino-like string pairs can be arranged, with repetition allowed, so the top and bottom concatenations match exactly.

Chomsky Hierarchy and Computability

- Chomsky Hierarchy of Grammars: The hierarchy organizes grammars and languages by how restrictive their production rules are.
- Unrestricted Grammars: An unrestricted grammar places almost no shape restriction on production rules.
- Context Sensitive Languages: A context-sensitive language needs more memory than one stack can provide, but its membership can be checked using space proportional to the input length.
- Relationship among language classes: Language classes form a ladder of increasing expressive power.
- Computability concepts: Computability asks whether any algorithm can solve a problem for every valid input.
- Primitive Recursive Functions: Primitive recursive functions are built from simple starting functions using composition and recursion over a natural-number argument.

Rapid definitions and notation

Language foundations

- Sigma: finite input alphabet; Sigma*: all finite strings including epsilon.
- Language: subset of Sigma*; epsilon is the empty string; empty-set is the empty language.
- Grammar G=(V,T,P,S): variables, terminals, productions, start symbol.
- Derivation S=>*w means zero or more rule applications.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Machine foundations

- DFA: one next state per state-symbol pair.
- NDFA: a set of possible next states; accept if some full path accepts.
- PDA: finite control plus stack; natural for nested structures.
- TM: finite control plus read/write tape; may halt, reject or run forever.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Proof and form

- Pumping: s=xyz, |xy|<=p, |y|>=1; pump i>=0.
- Myhill-Nerode: x~y iff every suffix distinguishes neither.
- CNF: A->BC or A->a, with start epsilon exception.
- GNF: A->aA1...Ak, with start epsilon exception.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

One-page construction and theorem checklist

Automata

- NDFA -> DFA: states are subsets; final iff subset intersects F.
- Epsilon-NFA: take epsilon-closure before and after symbol moves.
- DFA minimization: remove unreachable states, then refine partitions.
- Moore output is state-based; Mealy output is transition-based.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Regularity

- Closure: product for intersection; complete DFA then swap finals for complement.
- Pumping lemma is used to prove nonregularity; quantify over every split.
- Myhill-Nerode finite index iff regular; classes equal minimal DFA states.
- Arden: X=AX union B, epsilon not in A -> X=A*B.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Grammars and machines

- Remove useless variables by generating then reachable analyses.
- Remove unit chains by copying reachable non-unit productions.
- CFG <-> NPDA: derivation choices correspond to stack behavior.
- TM design: phases, markers, invariants, edge cases, correctness.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Language classes and memory models

Class

- Regular: DFA / NFA / regular expression / regular grammar.
- Context-free: CFG / nondeterministic PDA.
- Context-sensitive: noncontracting grammar / linear bounded automaton.
- Recursively enumerable: unrestricted grammar / Turing recognizer.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Containment

- Regular subset CFL subset CSL subset RE.
- The inclusions are proper in the standard hierarchy.
- Decidable languages are a proper subset of RE.
- A recognizer may loop on nonmembers; a decider halts on all inputs.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Separation examples

- {a^n b^n} is context-free and not regular.
- {a^n b^n c^n} is context-sensitive and not context-free.
- The Halting Problem is recognizable but undecidable.
- Finite-state memory is bounded; a PDA stack is unbounded LIFO; a TM has writable work tape.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Frequently asked exam and viva questions

Module I

- Convert an NDFA to a DFA using subset construction.
- Build an automaton for a regular expression and justify the language.
- Convert an automaton to a regular expression using state elimination.
- State and apply Arden's Theorem.
- Compare Moore and Mealy machines; convert one to the other.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Module II

- Prove a language is nonregular using the pumping lemma.
- Use closure properties to prove regularity or nonregularity.
- Apply Myhill-Nerode to prove a minimum-state lower bound.
- Remove useless/unit productions and convert a CFG to CNF/GNF.
- Show a CFG is ambiguous using two parse trees.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Module III

- Trace a PDA by configurations and state its acceptance mode.
- Explain CFG-PDA conversions and the meaning of stack markers.
- Design a TM for a specified language using phases and invariants.
- Prove the Halting Problem undecidable by diagonalization.
- Solve a PCP instance and explain why failure to find a match is inconclusive.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Practice set - easy to moderate

Easy

- 1. Over {0,1}, give a DFA for strings ending in 0.
- 2. Write an RE for strings containing at least one 1.
- 3. Give a right-linear grammar for a* b.
- 4. Trace a PDA for (), using stack bottom Z.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Moderate

- 5. Convert an NDFA with q0 -a->{q0,q1}, q1 -b->{q2} to a DFA by reachable subsets.
- 6. Minimize a DFA after marking distinguishable state pairs.
- 7. Show {a^n b^n} is not regular using the pumping lemma.
- 8. Remove A->B, B->C, C->c and preserve the language.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Answer checks

- 1. Two states: last symbol is 0 / otherwise; define epsilon as otherwise.
- 2. (0|1)*1(0|1)*.
- 3. S->aS | b generates a*b.
- 4. Push on '(', pop on ')'; accept only at end with bottom symbol.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Practice set - hard constructions

Proofs

- 1. Prove the language of binary strings whose length is a multiple of 4 is regular and give a minimal DFA.
- 2. Use Myhill-Nerode to show {a^n:n mod 5=0} needs at least five states.
- 3. Explain why one failed pumping decomposition does not prove nonregularity.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Conversions

- 4. Convert S->aSb|epsilon to CNF, showing nullable, terminal-helper and binarization stages.
- 5. Describe a CFG-to-PDA construction for balanced parentheses.
- 6. Give a TM design for {a^n b^n} with markers and a halting argument.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Reasoning hints

- 1. Use four residue states; distinguish each residue with a suffix.
- 2. Distinguish residues r and s using a suffix length that completes one residue only.
- 3. The lemma says every legal split must pump; one split is insufficient.
- 4-6. State each intermediate grammar/machine invariant and test epsilon/short inputs.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Worked answer cues for practice

Automata checks

- Ending in 0: start/non-0 state qN; on 0 go q0, on 1 go qN; q0 is final.
- Length mod 4: states r0..r3, transition r_i on either bit to r_(i+1 mod 4); only r0 final.
- Distinct residues have a suffix length that makes exactly one total length divisible by 4.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Grammar checks

- For a* b, S->aS | b; derivations contain zero or more a's then b.
- For balanced parentheses, push on open, pop on close, accept with no unmatched marker.
- For CNF, preserve S=>epsilon separately if needed; replace terminals in long RHS and binarize.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

TM / proof checks

- TM marker invariant: each completed sweep pairs one unmarked a with one unmarked b.
- Reject wrong order, missing partner, or extra unmarked b; accept when both blocks are exhausted.
- A decider must halt on all inputs; recognizer may loop on a nonmember.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Syllabus coverage and final self-check

Module I

- Finite Automata: all six listed topics.
- Regular Expressions: all four listed topics.
- Introduction to Machines: all five listed topics.
- Course wording retained: NDFA and ε-moves.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Module II

- Properties of Regular Sets: all six listed topics.
- Grammars: all nine listed topics.
- CNF/GNF and grammar reduction are separated into dedicated sections.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Module III

- Pushdown Automata: all four listed topics.
- Turing Machines: all five listed topics.
- Chomsky Hierarchy and Computability: all six listed topics.
- Outside-syllabus topics are not presented as syllabus content.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Recommended references listed in the syllabus

Books

- Introduction to Automata Theory, Languages, and Computation, Pearson Education, 3rd Edition, 2008.
- Theory of Computer Science: Automata, Languages and Computation, PHI Learning, 3rd Edition, 2012.
- An Introduction to Formal Languages and Automata, Jones & Bartlett Learning, 6th Edition, 2016.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Books continued

- Introduction to Languages and the Theory of Computation, McGraw Hill Education, 5th Edition, 2010.
- Elements of the Theory of Computation, Pearson Education, 2nd Edition, 2005.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Online resources listed

- NPTEL Theory of Computation Courses.
- GeeksforGeeks Automata Tutorials.
- TutorialsPoint Automata Theory.
- Programiz Computer Science Tutorials.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

One-minute oral revision

Explain

- A regular language is one some finite automaton recognizes.
- A CFG rewrites one variable; a PDA uses a stack to recognize its language.
- A TM has a read/write tape and models general effective computation.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Prove

- Pumping lemma: assume regular, choose a long witness, handle all legal splits.
- Myhill-Nerode: distinguish prefixes by a suffix; classes count minimum states.
- Halting: assume a decider, diagonalize on a machine's own encoding.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Design

- For each construction, state start, transition/action, and acceptance condition.
- Trace at least one accepted and one rejected boundary case.
- For grammar normal forms, verify every rule's shape after conversion.

Study check: explain this column aloud, then reproduce its key definition or construction from memory.

Course objectives and outcomes

Course objectives

- Learn the fundamentals of Regular and Context Free Grammars and Languages.
- Understand the relationship between Regular Languages and Finite Automata.
- Learn how to design automata and machines as acceptors, verifiers, and translators.
- Understand the relationship between Context Free Languages, Pushdown Automata, and Turing Machines.
- Learn how to design Pushdown Automata and Turing Machines for computation and language recognition.

Course outcomes

- Define the mathematical principles behind theoretical computer science.
- Identify different computational problems and their associated complexity.
- Differentiate between Finite Automata, Pushdown Automata, Linear Bounded Automata, and Turing Machines.
- Apply techniques for designing grammars and recognizers for programming languages.
- Correlate automata concepts with real-world computational applications.